

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт
Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №2
Инструменты для обработки больших данных
тема

Преподаватель

подпись, дата

С.С. Булыкин
инициалы, фамилия

Студент КИ23-17/1Б, 032323896
номер группы, зачетной книжки

подпись, дата

Ф.В. Фесенко
инициалы, фамилия

Красноярск 2026

1 Цель

Знакомство с инструментами для организации распределенной обработки данных, получение практических навыков по работе с Hadoop и подходом MapReduce.

2 Задачи

Решение задачи представляет собой получение четырёх пользователей, выполнивших максимальное количество одинаковых запросов. Выполнение практической работы предполагает решение следующих задач.

1. Реализовать первый шаг обработки данных – подсчет количества одинаковых запросов от каждого пользователя. Для преобразования каждой строки реализовать mapper, для объединения результатов – reducer.
2. Реализовать второй шаг обработки данных – нахождение требуемых четырех пользователей. Должно быть указано имя пользователя, тип запроса и количество. Для сортировки результатов предыдущего шага реализовать reducer.
3. Установить Apache Hadoop и запустить обработку файла в соответствии с вариантом задания.
4. В отчете привести результаты работы каждого шага.

3 Ход работы

Для выполнения работы Hadoop был развернут в Docker-контейнере. Это дает изолированное окружение, воспроизводимость и не засоряет основную систему.

Далее был написан первый mapper для подсчета количества одинаковых запросов от каждого пользователя. Mapper читает одну строку лога, извлекает из нее имя пользователя и сам запрос с помощью регулярных выражений, после чего выдает промежуточную пару вида ключ-значение, где ключом является комбинация: пользователя, запрос, а значением – единица. Таким образом,

маппер не занимается подсчетом, лишь размечает каждую строку. Код маппер-а представлен на рисунке 1.

```
(base) fedor@Mac scripts % cat mapper1.py
#!/usr/bin/env python3
import sys
import re

def parse_line(line):
    m = re.match(r'^(\S+)\s+--\s+(\S+)\s+\s+[[([^\]]+)\]]\s+(\S+)\s+(\S+)', line)
    if not m:
        return None
    return m.group(2), f"{m.group(4)} {m.group(5)}"

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    parsed = parse_line(line)
    if not parsed:
        continue
    user, request = parsed
    print(f"{user}|{request}\t1")
(base) fedor@Mac scripts %
```

Рисунок 1 – Код первого маппер-а

Далее был написан первый reducer, который выполняет подсчет. Hadoop гарантирует, что все пары с одинаковым ключом попадут в один reducer и придут туда последовательно, отсортированные по ключу. Это позволяет reducer-у работать без хэш-таблиц, просто идя по данным и сравнивая соседние элементы. Код первого reducer-а представлен на рисунке 2.

```
(base) fedor@Mac scripts % cat reducer1.py
#!/usr/bin/env python3
import sys

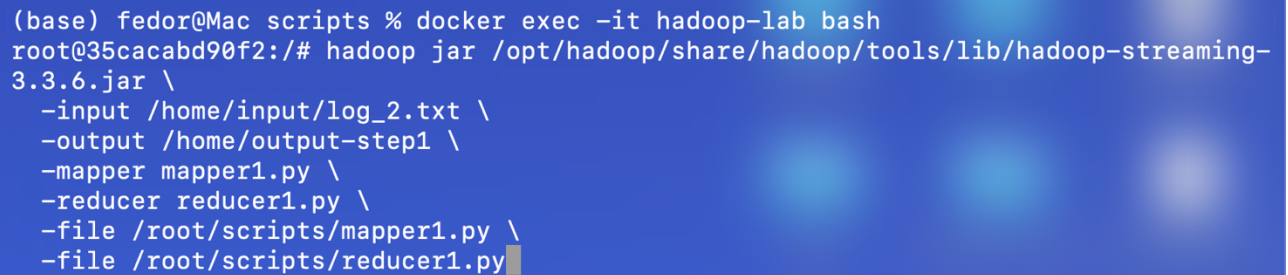
current_key = None
current_count = 0

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    try:
        key, count = line.split('\t', 1)
        count = int(count)
    except ValueError:
        continue
    if key == current_key:
        current_count += count
    else:
        if current_key is not None:
            print(f"{current_key}\t{current_count}")
            current_key = key
            current_count = count

if current_key is not None:
    print(f"{current_key}\t{current_count}")
(base) fedor@Mac scripts %
```

Рисунок 2 – Код первого reducer-а

Запуск первого шага проводился через утилиту Hadoop-streaming, входящую в состав Hadoop. Эта утилита позволяет в качестве mapper-а и reducer-а использовать произвольные исполняемые файлы – в нашем случае Python-скрипты, которые обмениваются данными с Hadoop через стандартные потоки ввода-вывода. После перехода в контейнер была запущена команда, представленная на рисунке 3.

A screenshot of a terminal window with a blue background. The text is white. It shows a user running a Docker command to execute a Hadoop streaming job. The command is: `docker exec -it hadoop-lab bash`. The prompt is `root@35cacabd90f2:/#`. The user enters `hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \` followed by several options: `-input /home/input/log_2.txt \`, `-output /home/output-step1 \`, `-mapper mapper1.py \`, `-reducer reducer1.py \`, `-file /root/scripts/mapper1.py \`, and `-file /root/scripts/reducer1.py`.

```
(base) fedor@Mac scripts % docker exec -it hadoop-lab bash
root@35cacabd90f2:/# hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-
3.3.6.jar \
-input /home/input/log_2.txt \
-output /home/output-step1 \
-mapper mapper1.py \
-reducer reducer1.py \
-file /root/scripts/mapper1.py \
-file /root/scripts/reducer1.py
```

Рисунок 3 – Команда для запуска первого шага обработки

Входной файл размером почти 2 ГБ был разбит Hadoop-ом на 14 блоков, каждый из которых обрабатывался отдельным mapper-ом. После завершения процесса обработки счетчики показали, что на вход поступило почти 19.5 млн строк, mapper-ы создали столько же промежуточных записей, а reducer обработал 36 уникальных групп (пар «пользователь – запрос»). Результат выполнения представлен на рисунке 4.

User0	GET	/big_image.png	274363
User0	GET	/index.html	274662
User0	POST	/save	274308
User1	GET	/big_image.png	275437
User1	GET	/index.html	274821
User1	POST	/save	274631
User2	GET	/big_image.png	274505
User2	GET	/index.html	274675
User2	POST	/save	274924
User3	GET	/big_image.png	273572
User3	GET	/index.html	274528
User3	POST	/save	274950
User4	GET	/big_image.png	274631
User4	GET	/index.html	273714
User4	POST	/save	274832
User5	GET	/big_image.png	274947
User5	GET	/index.html	274235
User5	POST	/save	274604
User6	GET	/big_image.png	274959
User6	GET	/index.html	274997
User6	POST	/save	274104
User7	GET	/big_image.png	273237
User7	GET	/index.html	274012
User7	POST	/save	274610
User8	GET	/big_image.png	274177
User8	GET	/index.html	274734
User8	POST	/save	273695
User9	GET	/big_image.png	274786
User9	GET	/index.html	275862
User9	POST	/save	274213
admin	GET	/big_image.png	9670759
admin	GET	/index.html	275235
admin	POST	/save	274457
anonymous	GET	/big_image.png	274656
anonymous	GET	/index.html	273702
anonymous	POST	/save	274876

Рисунок 4 – Результат первого шага обработки

Далее был написан второй mapper, задача которого – перегруппировать данные, полученные на первом шаге, таким образом, чтобы все запросы одного пользователя оказались под общим ключом. Если на первом шаге ключом была пара «пользователь|запрос», то теперь ключом становится только имя пользователя, а значением – строка вида «запрос:количество». Такая перегруппировка необходима для того, чтобы reducer на следующем этапе смог увидеть все запросы одного пользователя одновременно и выбрать среди них самый частый. Дополнительно все пары выдаются под одним общим ключом top, чтобы гарантированно попасть в единственный reducer и иметь возможность глобальной сортировки. Код второго маппера представлен на рисунке 5.

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    try:
        key, count = line.split('\t', 1)
        user, request = key.split('|', 1)
        count = int(count)
    except ValueError:
        continue
    print(f"top\t{user}|{request}|{count}")%
```

Рисунок 5 – Код второго mapper-а

Далее был написан второй reducer, который выполняет сразу три действия: находит для каждого пользователя его самый частый запрос, сортирует пользователей по количеству повторений этого запроса и выбирает первые четыре строки. Так как все данные приходят под одним ключом, reducer получает их в одном месте и может провести глобальную сортировку. Для поиска лучшего запроса каждого пользователя используется хэш-таблица, в которой накапливается максимальное значение count для каждого имени. После обработки всех входных строк таблица преобразуется в список, сортируется по убыванию количества и обрезается до первых четырёх элементов. Код второго reducer-а представлен на рисунке 6.

```
#!/usr/bin/env python3
import sys
from collections import defaultdict

best = defaultdict(lambda: (None, 0))

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    try:
        _, value = line.split('\t', 1)
        user, request, count = value.split('|')
        count = int(count)
    except ValueError:
        continue
    if count > best[user][1]:
        best[user] = (request, count)

results = sorted(
    [(count, user, request) for user, (request, count) in best.items()],
    reverse=True
)

for count, user, request in results[:4]:
    print(f"{user}\t{request}\t{count}")
```

Рисунок 6 – Код второго reducer-a

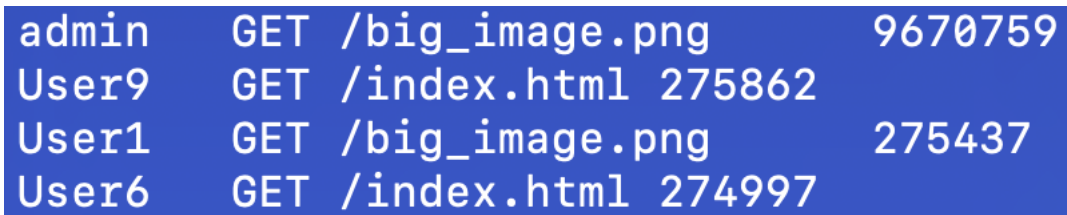
Запуск второго шага также проводился через утилиту Hadoop-streaming. В качестве входа использовалась директория, полученная на первом шаге. Команда запуска представлена на рисунке 7.

```
(base) fedor@Mac TAD2 % docker exec -it hadoop-lab bash
root@35cacabd90f2:/# hadoop jar /opt/hadoop/share/hadoop/tools/lib/hadoop-streaming-
3.3.6.jar \
-input /home/output-step1 \
-output /home/output-step2 \
-mapper mapper2.py \
-reducer reducer2.py \
-file /root/scripts/mapper2.py \
-file /root/scripts/reducer2.py
```

Рисунок 7 – Команда для запуска второго шага обработки

На вход второго шага поступило 36 строк, на выходе получилось ровно 4 – это подтверждается счетчиком Reduce output records = 4. Финальный результат представляет собой ответ на поставленную задачу: четыре пользователя с указанием их самого частого запроса и количества его повторений. Первое место ожидаемо занял пользователь admin с запросом GET /big_image.png и почти десятью миллионами повторений, остальные три позиции распределились между пользователями, у которых количество повторений лучшего запроса

находится в диапазоне 274–276 тысяч. Результат работы второго шага представлен на рисунке 8.



admin	GET	/big_image.png	9670759
User9	GET	/index.html	275862
User1	GET	/big_image.png	275437
User6	GET	/index.html	274997

Рисунок 8 – Результат второго шага обработки

4 Выводы

В ходе работы был развёрнут Single-Node кластер Apache Hadoop в Docker-контейнере, настроены HDFS, а также реализован двухшаговый MapReduce-конвейер на Python через утилиту Hadoop-streaming. На первом шаге было подсчитано количество одинаковых запросов от каждого пользователя, на втором — для каждого пользователя найден его самый частый запрос, выполнена сортировка и выбраны четыре лидирующие позиции. Работа позволила на практике познакомиться с архитектурой Hadoop, принципами MapReduce и механикой запуска задач через Hadoop-streaming с использованием Python-скриптов в качестве mapper-ов и reducer-ов.